

Software Handbuch

SILworX Plug-In O_Scope

```
#----- OPERATOR CLASSES -----
# Mirror Tool
class MirrorX(bpy.types.Operator):
    """This adds an X mirror to the selected object"""
    bl_idname = "object.mirror_mirror_x"
    bl_label = "Mirror X"

    @classmethod
    def poll(cls, context):
        return context.active_object is not None
        mirror_mod = modifier_ob.modifiers["Mirror"]

    # set mirror object to mirror_ob
    mirror_mod.mirror_object = mirror_ob

    if _operation == "MIRROR X":
        mirror_mod.use_x = True
        mirror_mod.use_y = False
        mirror_mod.use_z = False
    elif _operation == "MIRROR Y":
        mirror_mod.use_x = False
        mirror_mod.use_y = True
        mirror_mod.use_z = False
    elif _operation == "MIRROR Z":
        mirror_mod.use_x = False
        mirror_mod.use_y = False
        mirror_mod.use_z = True

    #selection at the end -add back the modifier
    mirror_ob.select= 1
    modifier_ob.select= 1
    bpy.context.select= 1
    print("Selected" + str(modifier_ob))
    #mirror_ob.select = 0
    #bpy.data.objects[selected_objects[0]].select = 1
except:
    print("please select exactly two objects")

#----- OPERATOR CLASSES -----
# Mirror Tool
class MirrorX(bpy.types.Operator):
    """This adds an X mirror to the selected object"""
    bl_idname = "object.mirror_mirror_x"
    bl_label = "Mirror X"

    @classmethod
    def poll(cls, context):
        return context.active_object is not None
        mirror_mod = modifier_ob.modifiers["Mirror"]

    # set mirror object to mirror_ob
    mirror_mod.mirror_object = mirror_ob

    if _operation == "MIRROR X":
        mirror_mod.use_x = True
        mirror_mod.use_y = False
        mirror_mod.use_z = False
    elif _operation == "MIRROR Y":
        mirror_mod.use_x = False
        mirror_mod.use_y = True
        mirror_mod.use_z = False
    elif _operation == "MIRROR Z":
        mirror_mod.use_x = False
        mirror_mod.use_y = False
        mirror_mod.use_z = True

    #selection at the end -add back the modifier
    mirror_ob.select= 1
    modifier_ob.select= 1
    bpy.context.select= 1
    print("Selected" + str(modifier_ob))
    #mirror_ob.select = 0
    #bpy.data.objects[selected_objects[0]].select = 1
except:
    print("please select exactly two objects")
```

Alle in diesem Handbuch genannten HIMA Produkte sind mit dem Warenzeichen geschützt. Dies gilt ebenfalls, soweit nicht anders vermerkt, für weitere genannte Hersteller und deren Produkte.

HIQuad®, HIQuad® X, HIMax®, HIMatrix®, SILworX®, XMR®, HICore® und FlexSILon® sind eingetragene Warenzeichen der HIMA Paul Hildebrandt GmbH.

Alle technischen Angaben und Hinweise in diesem Handbuch wurden mit größter Sorgfalt erarbeitet und unter Einschaltung wirksamer Kontrollmaßnahmen zusammengestellt. Bei Fragen bitte direkt an HIMA wenden. Für Anregungen, z. B. welche Informationen noch in das Handbuch aufgenommen werden sollen, ist HIMA dankbar.

Technische Änderungen vorbehalten. Ferner behält sich HIMA vor, Aktualisierungen des schriftlichen Materials ohne vorherige Ankündigungen vorzunehmen.

Für registrierte Kunden stehen die Produktdokumentationen im HIMA Extranet als Download zur Verfügung.

© Copyright 2023, HIMA Paul Hildebrandt GmbH

Alle Rechte vorbehalten.

Kontakt

HIMA Paul Hildebrandt GmbH

Postfach 1261

68777 Brühl

Tel.: +49 6202 709-0

Fax: +49 6202 709-107

E-Mail: info@hima.com

Revisions- index	Änderungen	Art der Änderung	
		technisch	redaktionell
1.0	Erstausgabe	X	X

Inhaltsverzeichnis

1	SILworX Plug-Ins (lizenzpflichtige Option)	5
2	Plug-Ins konfigurieren	6
2.1	SSL-Zertifikat und -Schlüssel erstellen	6
2.2	«settings.ini» vervollständigen	7
3	Plug-In-Registrierung	9
3.1	Vermeidung von Fehlern bei der Plug-In-Registrierung	9
4	Plug-Ins starten	10
4.1	Plug-In Hash-Dateien	10
4.2	«customplugins.json» vervollständigen	11
5	Funktionsweise des Plug-Ins «O_Scope.exe»	12
5.1	Schaltflächen und Eingabefelder	15
5.2	Schnell Tasten und Tastenkombinationen	17

1 SILworX Plug-Ins (lizenzpflichtige Option)

Zur Erweiterung der Funktionalität kann SILworX ab Version 14 mit Plug-Ins ausgestattet werden. Plug-Ins sind nicht-sichere Anwendungen, die mit SILworX interagieren, aber nicht in der SILworX Entwicklungsumgebung enthalten sind und auch nicht mit SILworX zertifiziert werden.

Mit Plug-Ins sind Anwender in der Lage, zusätzliche Funktionen nach eigenem Bedarf zu erstellen oder auch für Dritte bereitzustellen. Voraussetzung ist das Vorhandensein der erforderlichen Lizenzen. Anwenderseitig erstellte Plug-Ins sind mit der Plug-In Feature-Lizenz in Kombination mit der SILworX API-Lizenz verwendbar. Für von HIMA bereitgestellte, lizenzfreie Plug-Ins wird nur die SILworX Lizenz benötigt.

Plug-Ins laufen als eigene Prozesse in einem von SILworX getrennten Speicherbereich und interagieren mit SILworX über die Schnittstellen *SILworX API* und *WebSocket*. Plug-Ins lassen sich rückwirkungsfrei einbinden und wirken sich nicht negativ auf die sicheren SILworX Funktionen aus.

Da Plug-Ins keine integralen Bestandteile von SILworX sind, müssen sie in der Datei *settings.ini* konfiguriert werden. Dafür existiert in der *settings.ini* die Kategorie *[Plugin_Server]*. SILworX prüft beim Starten das Vorhandensein der registrierten Plug-Ins und stellt deren Funktionalität in SILworX bereit. SILworX startet die in der *settings.ini* unter *PluginNames* aufgelisteten Plug-Ins automatisch. Die Reihenfolge der Plug-In-Namen bestimmt die Abarbeitungspriorität der Plug-Ins im Falle gleichzeitiger Triggerungen.

Plug-Ins werten von SILworX bereitgestellte vordefinierte Trigger und anwenderspezifische Trigger aus und führen die vom Anwender gewünschten Funktionen aus. Die von Plug-Ins in SILworX ausgelösten Aktionen werden im Logbuch protokolliert. In der Statuszeile werden rechts neben den API-Informationen in einem eigenen Bereich Informationen zum Plug-In-Feature angezeigt.

Die gestarteten Plug-Ins haben keine eigene Benutzerverwaltung. Die Authentifizierung erfolgt durch das Öffnen des Projekts in SILworX. Die dabei erzeugte User-Session steht auch den Plug-Ins und der SILworX API zur Verfügung.

Wenn das Plug-In-Feature gestartet ist, können Sie im *Hilfe*-Menü von SILworX über die Menüfunktion *Plug-In-Informationen* das entsprechende Dialogfenster öffnen. Das Dialogfenster enthält die beiden Listen *Konfigurierte Plug-Ins* und *Registrierte Plug-Ins*. Zusätzlich wird angezeigt, mit welchen Aktionen ein Plug-In in SILworX verknüpft ist und welche Trigger aus SILworX bzw. nach SILworX registriert sind.

Zusammenfassung:

- Plug-Ins sind SILworX Erweiterungen, die von Anwendern selbst erstellt werden können.
- Plug-Ins basieren auf Programmen oder Scripten, ggf. mit Zusatzdateien, die in SILworX konfiguriert werden.
- Plug-Ins sind eigenständige Module, die in eigenen Prozessen laufen.
- Plug-Ins werden automatisch mit SILworX gestartet.
- Plug-Ins haben über die SILworX API-Funktionen Zugriff auf SILworX.
- Plug-Ins benötigen eine Plug-In-Feature-Lizenz und eine SILworX API-Lizenz.
- Plug-Ins können auf vordefinierte Trigger und benutzerdefinierte Trigger reagieren.
- Das Ausführen von Plug-Ins wird in der SILworX Statusleiste angezeigt.

2 Plug-Ins konfigurieren

Damit Sie SILworX Plug-Ins verwenden können, müssen Sie die gewünschten Plug-Ins auf einem für SILworX zugänglichen Laufwerk speichern und in der Datei *settings.ini* konfigurieren. Zusätzlich benötigen Sie eine Plug-In-Feature-Lizenz, eine SILworX API-Lizenz und ein SILworX API-Zertifikat. Zugriffe auf die SILworX API sind ausschließlich über SSL (TLS 1.2 oder TLS 1.3) möglich.

Die Verwendung von SSL setzt ein gültiges Zertifikat und den zugehörigen privaten Schlüssel voraus. Die Pfade zu diesen Dateien müssen in der *settings.ini* in der Kategorie *[API_Server]* eingetragen werden. Damit Plug-Ins in Verbindung mit der SILworX API verwendet werden können, muss der Konfigurationsparameter *Enabled* der Kategorie *[API_Server]* in der *settings.ini* auf TRUE gesetzt werden.

SILworX liest beim Start diverse Plug-In-Konfigurationsparameter aus der *settings.ini* ein, die in der Kategorie *[Plugin_Server]* zusammengefasst sind. Danach prüft SILworX die Integrität der registrierten Plug-Ins anhand von Hash-Codes. Ist das Ergebnis der Integritätsprüfung positiv, stellt SILworX die Funktionen der Plug-Ins bereit.

Um Plug-Ins in SILworX nutzen zu können, müssen die Namen der Plug-Ins in der Liste *PluginNames*, durch Kommata getrennt, vorhanden sein. Wenn mehrere Plug-Ins auf einen bestimmten Trigger reagieren, bestimmt die Reihenfolge in der *PluginNames*-Liste die Ausführungsreihenfolge der Plug-Ins.



Maximal können 254 Plug-Ins registriert werden.

Sollten in *PluginNames* Plug-In-Namen mehrfach vorkommen, so werden die Duplikate beim Einlesen der Plug-In-Konfiguration verworfen und SILworX meldet einen Fehler.



Die Plug-In-Namen werden in den Meta-Daten der Plug-Ins definiert. Sie können unterschiedlich zu den verwendeten Dateinamen der Plug-Ins sein.

Die in *PluginNames* gelisteten Plug-Ins müssen als ausführbare Programme (*.exe) vorliegen und zum Schutz vor unerwünschten Manipulationen muss jedes dieser Plug-Ins in der Hash-Datei *customplugins.json* konfiguriert sein.



SILworX installiert einige von HIMA bereitgestellte Plug-Ins und trägt deren Namen standardmäßig als *PluginNames* ein. Diese Liste können Sie an beliebiger Position um Ihre eigenen *PluginNames* ergänzen. Auch für die Verwendung der HIMA Plug-Ins muss der API-Server **Enabled** sein.

SILworX startet nach der Initialisierung alle in *PluginNames* konfigurierten Plug-Ins automatisch. Dazu müssen für die in *PluginNames* gelisteten Plug-Ins die passenden Hash-Codes in der *customplugins.json* existieren. Die Dateipfade werden ebenfalls der *customplugins.json* entnommen. SILworX gibt im Logbuch Meldungen aus, wenn Plug-Ins nicht gestartet werden können.

2.1 SSL-Zertifikat und -Schlüssel erstellen

Um die SILworX API nutzen zu können, benötigen Sie ein SSL-Zertifikat im PEM-Format, welches Sie selbst erstellen. Zu diesem Zertifikat benötigen Sie außerdem den zugehörigen privaten Schlüssel im PEM-Format, der mit dem RSA-Algorithmus ohne Passphrase erstellt wurde.

Zertifikat und Schlüssel müssen in der SILworX settings.ini eingetragen werden.



Gültigkeitsdauer des SSL-Zertifikats beachten!

Wenn Sie ein neues SSL-Zertifikat erzeugen, können Sie dessen Gültigkeitsdauer in Tagen definieren. HIMA empfiehlt, die Gültigkeitsdauer auf 365 Tage oder weniger zu beschränken. Einige Clients (z. B. aktuelle Browser) akzeptieren keine selbstsignierten Zertifikate mit Gültigkeitsdauer größer 365 Tage.

So erzeugen Sie ein gültiges Zertifikat:

- Wählen Sie in SILworX den Menüpunkt **Extras, SILworX-API Zertifikat erstellen**. Der Dialog *SILworX-API Zertifikat erstellen* öffnet sich.
- Geben Sie im Feld *Dateipfad - Zertifikat* den Pfad für das Zielverzeichnis und den Zertifikatsnamen ein oder verwenden Sie die Schaltfläche rechts neben dem Feld *Dateipfad - Zertifikat*, um das Zielverzeichnis und den Zertifikatsnamen auszuwählen.
- Geben Sie im Feld *Dateipfad - Schlüssel* den Pfad für das Zielverzeichnis und den Schlüsselnamen ein oder verwenden Sie die Schaltfläche rechts neben dem Feld *Dateipfad - Schlüssel*, um das Zielverzeichnis und den Schlüsselnamen auszuwählen.
- Füllen Sie bei Bedarf die optionalen Felder mit Ihren Daten aus.
- Bestätigen Sie Ihre Eingaben mit **Erstellen**.

2.2

«settings.ini» vervollständigen

Zum Konfigurieren der SILworX API und der Plug-Ins müssen Sie die nachfolgenden Einträge in der Datei *settings.ini* ergänzen.

- Öffnen Sie die Datei *settings.ini* im SILworX Programmdaten-Verzeichnis %ProgramData%\SILworX_vXX.X.0 Rxxxx\settings\, z. B. C:\ProgramData\SILworX_v14.0.0 R2908\settings\settings.ini, mit einem Texteditor.
- Suchen Sie in der Datei *settings.ini* nach dem Eintrag *[API_Server]*. Existiert dieser Eintrag nicht, fügen Sie ihn am Ende der Datei an.
- Unterhalb von *[API_Server]* werden die nachfolgend beschriebenen Konfigurationsparameter benötigt, die Sie bei Bedarf ergänzen müssen.
`Enabled=true`
`IpAddress=::1`
`ServerCertificate=Absoluter Pfad auf die Zertifikatsdatei`
`ServerPrivateKey=Absoluter Pfad auf die Schlüsseldatei`
- Suchen Sie in der Datei *settings.ini* nach dem Eintrag *[Plugin_Server]*. Existiert dieser Eintrag nicht, fügen Sie ihn am Ende der Datei an.
- Unterhalb von *[Plugin_Server]* werden die nachfolgend beschriebenen Konfigurationsparameter benötigt, die Sie bei Bedarf ergänzen müssen
`PluginNames=Liste der Namen der Plug-Ins`



API-Requests über localhost

Plug-Ins verwenden den Domainnamen *localhost* zur Kommunikation mit der SILworX API. Das Betriebssystem löst *localhost* zuerst nach IPv6 auf, bevor eine IPv4-*Localhost*-Adresse zum Tragen kommt. Dadurch kann es bei jedem API-Aufruf zu Verzögerungen bei der Ausführung im Bereich einiger Sekunden kommen. Als Abhilfe sollte der SILworX API-Server auf die IPv6-*Localhost*-Adresse konfiguriert werden. Dazu ist in der *settings.ini* der Parameter `IpAddress=::1` einzutragen.

Beispiel für die «settings.ini»

```
...  
[API_Server]  
Enabled=true  
IpAddress=::1  
ServerCertificate=C:\Plugins\Certificates\api_cert.pem  
ServerPrivateKey=C:\Plugins\Certificates\api_key.pem  
[Plugin_Server]  
PluginNames=hima.ssl_certificates_creator, o-scope
```

3 Plug-In-Registrierung

Mit der Registrierung, die nach dem erfolgreichen WebSocket-Verbindungsaufbau erfolgt, melden sich Plug-Ins bei SILworX mit ihrem Namen, den Metadaten und ihren gewünschten Triggern an. Dazu müssen die Plug-Ins mit ihren Namen in der *settings.ini* korrekt konfiguriert sein.

Plug-Ins registrieren sich nach ihrem Start mittels Nachrichten, die sie an SILworX senden. War die Registrierung erfolgreich, werden im Logbuch Meldungen mit den Dateipfaden der Plug-Ins ausgegeben. Zusätzlich werden die Hash-Codes der Plug-Ins angezeigt.

3.1 Vermeidung von Fehlern bei der Plug-In-Registrierung

War die Registrierung nicht erfolgreich, wird im Logbuch eine Fehlermeldung mit einem Hinweis auf die Ursache der fehlgeschlagenen Registrierung ausgegeben. Zur Fehlervermeidung beachten Sie die folgenden Hinweise:

- **Mehrfachregistrierung von Plug-Ins**

Im Konfigurationsparameter *PluginNames* der *settings.ini* dürfen keine doppelten Namen vorkommen.

- **Nichterlaubte Registrierung eines Plug-Ins**

Im Konfigurationsparameter *PluginNames* der *settings.ini* muss der in den Meta-Daten eines Plug-Ins definierte Plug-In Name aufgeführt sein. Dieser Plug-In Name muss auch dem Schlüssel *plugin_name* in der *customplugins.json* als Wert zugewiesen sein.

- **Maximale Anzahl von Plug-Ins**

Es können in SILworX maximal 254 Plug-Ins gleichzeitig registrieren werden.

- **Maximale Anzahl von Triggern**

Die maximale Anzahl von 20 Trigger-Einträgen für das *Extra-Menü* und 100 Trigger-Einträgen für die Summe aller Kontextmenüs darf nicht überschritten werden.

- **Falsche Formatierung der Hash-Datei**

Achten Sie auf die korrekte Anzahl der öffnenden und schließenden Klammern sowie der Anführungszeichen. Leerzeichen außerhalb von Zeichenketten sind nicht signifikant. Der Pfad zur Plug-In Datei muss Schrägstriche / oder doppelte Rückstriche \\ enthalten, einfache Rückstriche \ sind nicht zulässig.

4 Plug-Ins starten

Das Starten von Plug-Ins bedeutet, dass Plug-Ins zunächst in SILworX registriert werden, um mit SILworX interagieren zu können. Nach der erfolgreichen Plug-In-Registrierung befinden sich die Programme im Arbeitsspeicher des Rechners. Die eigentlichen Interaktionen mit SILworX und die Datenverarbeitung werden durch Trigger-Benachrichtigungen ausgelöst, die SILworX an die Plug-Ins sendet.

SILworX startet bei der Initialisierung einen Plug-In-Prozess für jedes der in der *settings.ini* unter *PluginNames* konfigurierten Plug-Ins, sofern die Plug-In-Daten in einer der Plug-In Hash-Dateien eingetragen sind. Existieren diese Daten nicht, gibt SILworX eine Meldung aus und startet das Plug-In nicht.

SILworX startet die in der *settings.ini* aufgeführten Plug-Ins automatisch. Beim Start eines Plug-Ins übergibt SILworX eine Kommandozeile mit einigen Parametern. Die Kommandozeile wird auch als Meldung im Logbuch ausgegeben.

Die gestarteten Plug-Ins haben keine eigene Benutzerverwaltung, die Benutzerauthentifizierung findet auf Projektebene statt. Durch das Öffnen eines Projekts mit Benutzerverwaltung authentifiziert sich der Anwender im SILworX Projekt. Die dabei erzeugte User-Session steht sowohl den Plug-Ins als auch der darin verwendeten SILworX API zur Verfügung.

4.1 Plug-In Hash-Dateien

Für die in der *settings.ini* unter *PluginNames* konfigurierten Plug-Ins prüft SILworX vor deren Registrierung die Echtheit der Plug-Ins. Die Prüfung erfolgt durch Vergleiche der aktuell berechneten Hash-Codes der Plug-Ins mit den vom Hersteller der Plug-Ins bereitgestellten Sollwerten, die zuvor in Hash-Dateien gespeichert wurden.

SILworX verwendet zwei unterschiedliche Hash-Dateien, eine für HIMA Plug-Ins und eine weitere für Anwender Plug-Ins. Die Hash-Datei für HIMA Plug-Ins enthält die Hash-Codes derjenigen Plug-Ins, die ohne Plug-In-Feature-Lizenz und ohne API-Lizenz genutzt werden können.

SILworX verwirft beim Einlesen der Plug-In-Konfiguration aus der *settings.ini* und den Hash-Dateien doppelt vorkommende Plug-In-Namen und meldet diese als Fehler.



Die Hash-Datei für HIMA Plug-Ins darf kundenseitig nicht verändert werden.

Die Hash-Datei *customplugins.json* enthält die Hash-Codes der Plug-Ins, für die eine Plug-In-Feature-Lizenz benötigt wird. Diese Hash-Datei muss kundenseitig angepasst werden. Die Datei enthält für jedes Plug-In einen separaten Datenblock und ist wie folgt aufgebaut:

```
{
  "plugins": [
    {
      "plugin_name": "Plug-In Name gemäß Meta-Daten des Plug-Ins 1",
      "plugin_path": "Absoluter Pfad auf das Plug-In 1",
      "plugin_hash": "Hash-Code nach SHA-256 Plug-In 1"
    },
    {
      "plugin_name": " Plug-In Name gemäß Meta-Daten des Plug-Ins 2",
      "plugin_path": "Absoluter Pfad auf das Plug-In 2",
      "plugin_hash": "Hash-Code nach SHA-256 Plug-In 2"
    }
  ]
}
```

4.2 «customplugins.json» vervollständigen

Zum Konfigurieren der Plug-Ins müssen Sie die nachfolgenden Einträge in der Datei *customplugins.json* ergänzen.

- Öffnen Sie die Datei *customplugins.json* im SILworX Programmdatei-Verzeichnis %ProgramData%\SILworX_vXX.X.0 Rxxxx\plugins\, z. B. C:\ProgramData\SILworX_v14.0.0 R2908\plugins\customplugins.json, mit einem Texteditor.
- Die Hash-Datei enthält für jedes Plug-In einen separaten Datenblock, bestehend aus einer in geschweiften Klammern eingeschlossenen Liste von Schlüssel : Werte-Paaren:

```
{  
  "plugin_name": "Plug-In Name gemäß den Meta-Daten des Plug-Ins",  
  "plugin_path": "Absoluter Pfad auf das Plug-In",  
  "plugin_hash": "Hash-Code nach SHA-256"  
}
```

Ergänzen Sie in der Datei *customplugins.json* diesen Datenblock mit den für das Plug-In passenden Werten. Existiert dieser Datenblock nicht, fügen Sie ihn nach der öffnenden eckigen Klammer ein. Beachten Sie, dass die Zeichenketten in doppelt geraden Anführungszeichen " gesetzt werden müssen.



Rückstriche \ in JSON-Zeichenketten

Entsprechend der JSON-Formatdefinition wird der Rückstrich \ in JSON-Zeichenketten als Maskierungszeichen zur Einleitung von Escape-Sequenzen verwendet. Daher müssen die Rückstriche in Pfadangaben entweder durch doppelte Rückstriche \\ oder durch Schrägstriche / ersetzt werden.

Beispiel für die «customplugins.json»

```
{  
  "plugins": [  
    {  
      "plugin_name": "o-scope",  
      "plugin_path": "C:\\\\Plugins\\\\O_Scope.exe",  
      "plugin_hash":  
        "1C4063E053D76D96A7F36F12F16759EF854BC89C80594453D5F8F22DD5FB98B2"  
    }  
  ]  
}
```

5 Funktionsweise des Plug-Ins «O_Scope.exe»

Das Plug-In wird über das Kontextmenü eines Programms gestartet:

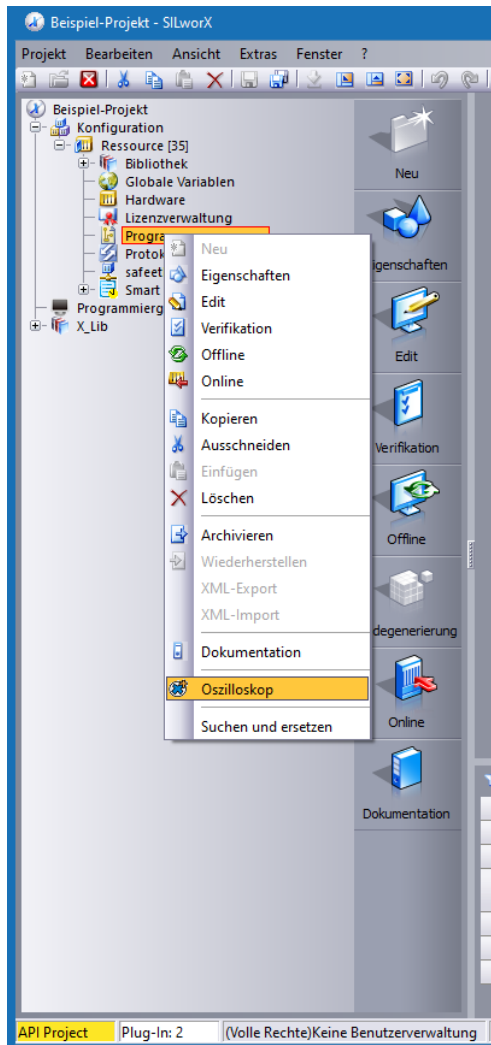


Bild 1: Programm-Kontextmenü mit Menüeintrag für das Plug-In

Das Plug-In führt einen System-Login auf die laufende Steuerung aus und liest für das im Strukturbaum ausgewählte Anwenderprogramm alle visualisierbaren lokalen Variablen aus. Die Variablen werden in einem Strukturbaum hierarchisch aufgelistet. Für vom Anwender ausgewählte Variablen werden deren aktuelle Prozesswerte zyklisch aus der Steuerung ausgelesen und als Funktion der Zeit grafisch dargestellt.

Dazu führt das Plug-In nach dem Start nacheinander die folgenden SILworX-Aktionen aus:

- Abrufen des Projekt-, Konfiguration-, Ressource- und Programmnamens aus dem Strukturbaum
- Ausführen eines System-Logins über das Menü **Extras, Online** der übergeordneten Ressource
- Abrufen der mittleren Zykluszeit aus dem Control Panel der verbundenen Ressource
- Abrufen aller lokalen Variablen über das Menü **Forcen, Force-Editor, Lokale Variablen** des markierten Programms
- Trennen der Online-Sitzung der verbundenen Ressource über das Menü **Online, Trennen**

Nach Auswahl der gewünschten Zeitdauer des Diagramms und der darzustellenden Variablen werden nach Betätigung der Schaltfläche **Starten** folgende SILworX-Aktionen ausgeführt:

- Ausführen eines System-Logins über das Menü **Extras, Online** des ausgewählten Programms
- Zyklisches Auslesen der Prozesswerte der selektierten programmlokalen Variablen bis zur Betätigung der Schaltfläche **Beenden** oder Schließen des Plug-Ins-Fensters
- Trennen der Online-Sitzung der verbundenen Ressource über das Menü **Online, Trennen**

Nach dem Start zeigt das Plug-In einen Dialog zur Abfrage der Zugangsdaten zur Steuerung.

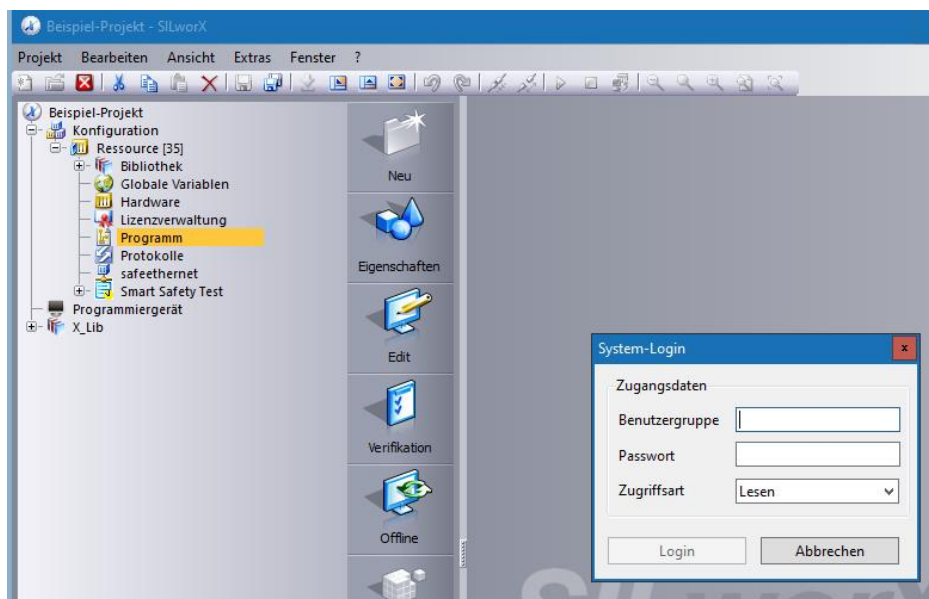


Bild 2: Anmeldedialog des Plug-Ins für den System-Login

Nach Eingabe der Zugangsdaten und Betätigung der Schaltfläche **Login** zeigt das Plug-In eine Meldung über das Abrufen der benötigten Daten aus der Steuerung.

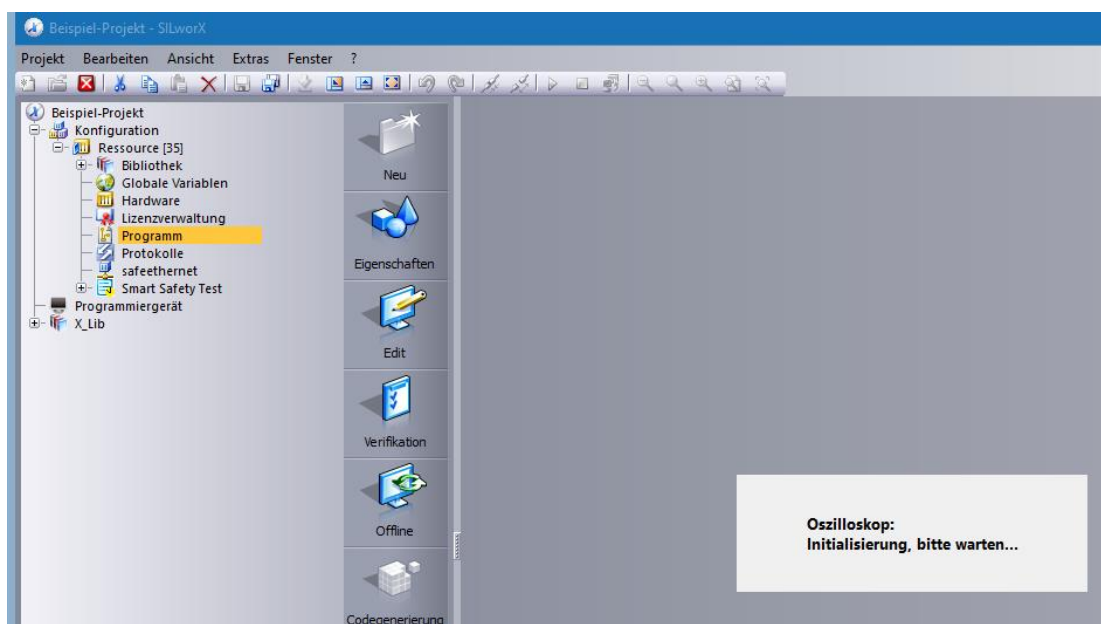


Bild 3: Initialisierung des Plug-Ins

Nachdem alle Daten abgerufenen sind, erscheint das Hauptfenster des Plug-Ins.

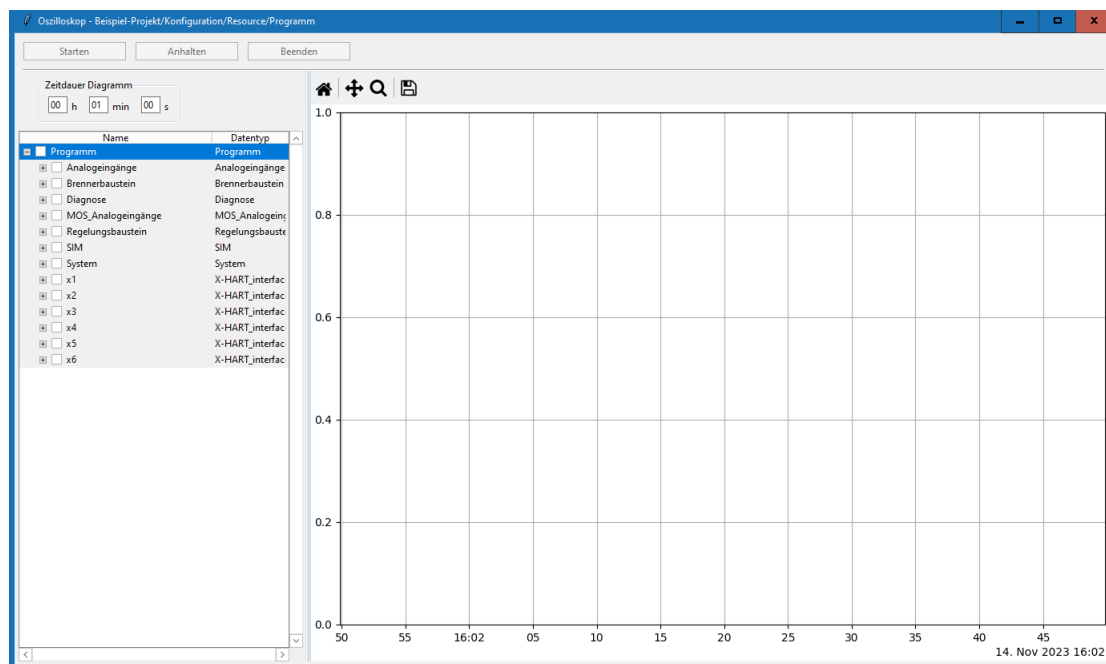


Bild 4: Hauptfenster des Plug-Ins mit Variablen-Auswahl

Es sind bis zu zehn Variablen aus dem Strukturbaum auswählbar, deren zeitlicher Verlauf dargestellt werden soll. Die Auswahl erfolgt über Klick in die Checkbox vor dem Variablennamen oder über Doppelklick in die entsprechende Zeile. Über die Eingabe **Zeitdauer Diagramm** wird die Länge des sichtbaren Kurvenverlaufs festgelegt. Nach Auswahl der darzustellenden Variablen und Festlegung der darzustellenden Zeitdauer wird die Aufnahme der Prozesswerte über die Schaltfläche **Starten** gestartet.



Variablen des Datentyps *TIME* werden als Sekundenwerte interpretiert, d.h. 1000 ms in SILworX entsprechen im Trend einem Wert von 1.0.

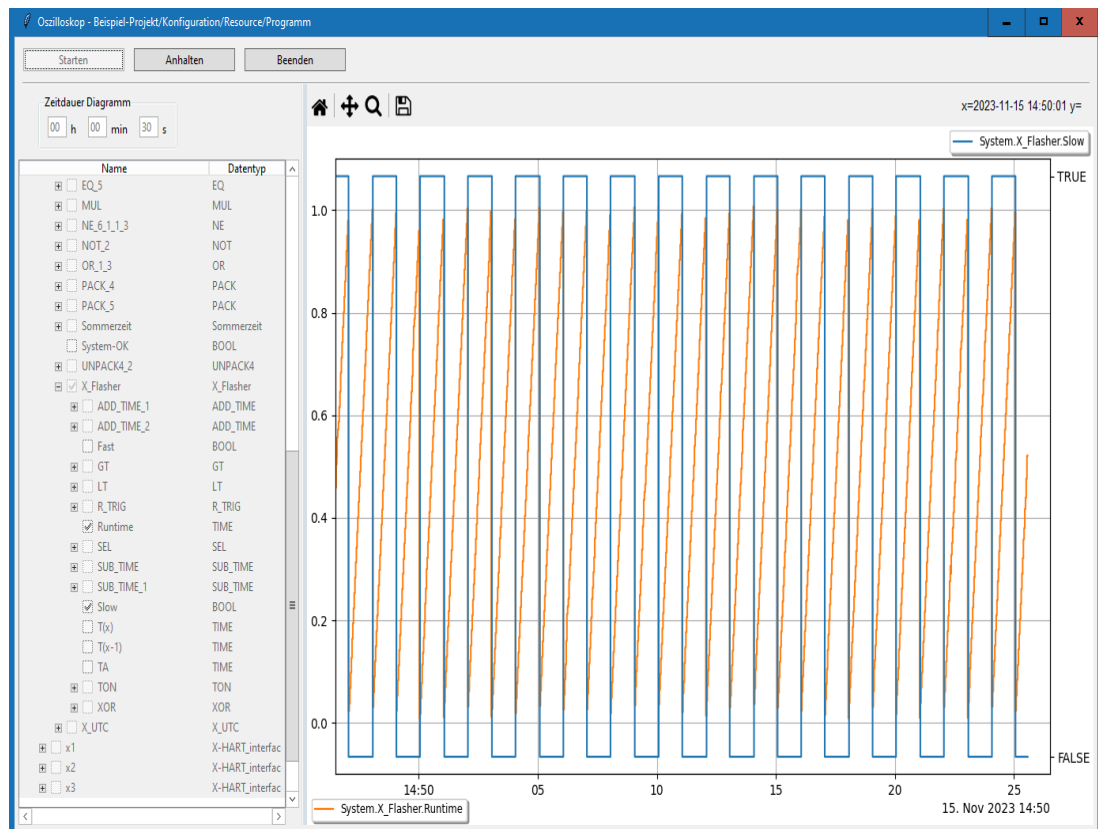


Bild 5: Grafische Visualisierung der Prozesswerte als Funktion der Zeit.

Die während der Laufzeit zyklisch ausgelesenen Prozesswerte werden von SILworX in zwei Binärdateien im Verzeichnis %Temp%\SILworX_Plugins zwischengespeichert. Die Abfrage der Prozesswerte erfolgt mit der doppelten mittleren Zykluszeit der Steuerung, aber schnellstens mit 30 ms. Bei einer langen darzustellenden Zeitdauer des Diagramms wird zur Begrenzung der handzuhabenden Datenmenge der Abfragezyklus u.U. vergrößert.

Das Plug-In erzeugt im Verzeichnis %AppData%\SILworX_Plugins\ eine Logdatei mit dem Namen «o-scope.log». Die Logdatei enthält alle aufgetretenen Status- und Fehlermeldungen mit Angabe des Zeitpunkts, zu dem die Meldung ausgegeben wurde. Die Ausgabe erfolgt in tabellarischer Form.

5.1 Schaltflächen und Eingabefelder

Starten, Anhalten, Fortsetzen, Beenden

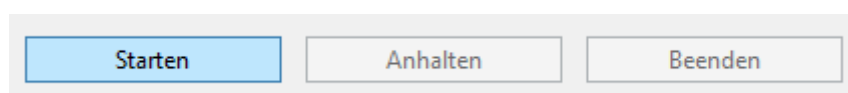
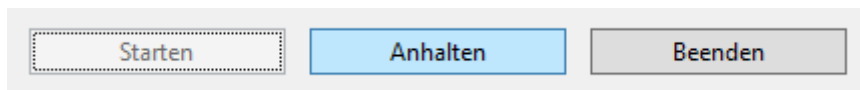
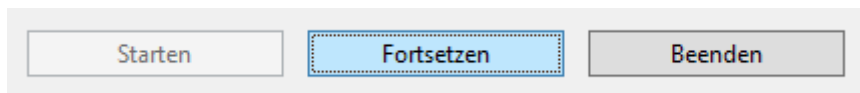
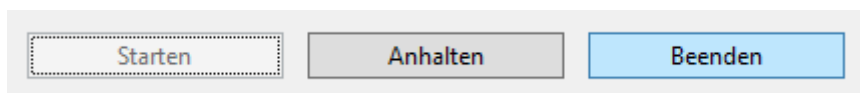


Bild 6: Schaltfläche **Starten**

Die Schaltfläche **Starten** wird freigegeben, sobald für die Zeitdauer Diagramm ein gültiger Wert eingetragen ist und mindestens eine Variable zur Darstellung ausgewählt wurde. Nach dem Starten ist bis zum Beenden der Aufzeichnung keine Änderung an der Zeitdauer Diagramm und der Variablenauswahl im Strukturbaum mehr möglich.

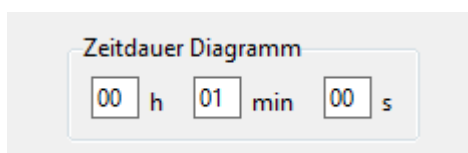
Bild 7: Schaltfläche **Anhalten**Bild 8: Schaltfläche **Fortsetzen**

Die Schaltfläche **Anhalten** ist nach dem Starten der Aufzeichnung verfügbar. Bei Betätigung wird die momentane Abbildung zur Erleichterung einer Auswertung nicht mehr aktualisiert. Die Abfrage der Prozesswerte aus der Steuerung läuft im Hintergrund weiter. Eine Änderung der eingestellten Aufzeichnungs-Parameter ist nicht möglich. Die Aktualisierung der Trendkurve wird nach Betätigung der Schaltfläche **Fortsetzen** an der dann aktuellen Stelle wieder aufgenommen.

Bild 9: Schaltfläche **Beenden**

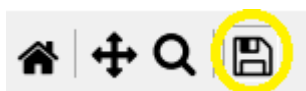
Die Schaltfläche **Beenden** wird direkt nach dem Starten der Aufzeichnung freigegeben. Nach dem Beenden der Aufzeichnung wird die Abfrage der Prozesswerte aus der Steuerung gestoppt und die Online-Sitzung mit der verbundenen Ressource getrennt. Die Zeitdauer Diagramm und der Strukturbaum zur Variablenauswahl nehmen wieder Eingaben des Anwenders entgegen.

Zeitdauer Diagramm

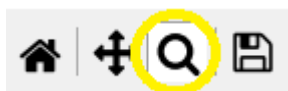
Bild 10: Eingabefelder **Zeitdauer Diagramm**

Diese Eingabefelder bestimmen die Zeitdauer des sichtbaren Trendkurve. Der kleinste zulässige Wert ist zehn Sekunden, der größte zulässige Wert sind fünf Stunden, 59 Minuten und 59 Sekunden. Erst nach Eingabe einer gültigen Zeitdauer kann die Schaltfläche **Starten** betätigt werden.

Schaltflächen im Trendfenster

Bild 11: Schaltfläche **Speichern**.

Die Schaltfläche **Speichern** öffnet einen Dialog zum Speichern der momentanen Abbildung in verschiedenen Grafikformaten.

Bild 12: Schaltfläche **Zoomen**.

Mit der Betätigung der Schaltfläche **Zoomen** wird die Funktion «In Rechteck zoomen» ein- oder ausgeschaltet. Bei eingeschalteter Funktion kann die Trendgrafik vergrößert oder verkleinert werden.

Bild 13: Schaltfläche **Verschieben**.

Mit der Betätigung der Schaltfläche **Verschieben** wird die Funktion «Achsen verschieben» ein- oder ausgeschaltet. Bei eingeschalteter Funktion kann die Trendgrafik ohne Zoomen auf der X- und der Y-Achse verschoben werden.

Bild 14: Schaltfläche **Defaultansicht**.

Mit der Betätigung der Schaltfläche **Defaultansicht** wird nach einem Zoom oder einer Verschiebung die ursprüngliche Ansicht wiederhergestellt werden.

5.2 Schnell Tasten und Tastenkombinationen

Es sind die in SILworX üblichen Schnell Tasten und Tastenkombinationen verfügbar:

System-Login

Schnell taste, Kombination	Beschreibung
Strg+A	Füllt Benutzergruppe, Passwort und Zugriffsart mit der werksseitigen Standardeinstellung für <Administrator> aus

Eingabefelder Zeitdauer Diagramm

Schnell taste, Kombination	Beschreibung
Tab	Bewegt die Markierung um eine Zelle nach rechts
Umschalt taste+Tab	Bewegt die Markierung um eine Zelle nach links

Strukturbaum

Schnell taste, Kombination	Beschreibung
----------------------------	--------------

↑ ↓ ← →	Bewegt die Markierung um ein (sichtbares) Element nach oben, unten, links oder rechts
+, -, Eingabetaste	Klappt den Strukturbaum auf oder zu
Leertaste	Klappt den Strukturbaum auf oder zu Aktiviert oder deaktiviert das Kontrollkästchen
Strg+Pos1	Bewegt die Markierung an den Anfang des Strukturbaums
Strg+Ende	Bewegt die Markierung an das Ende des Strukturbaums
Buchstabe	Bewegt die Markierung auf das nächste Element auf der gleichen Hierarchie-Ebene, welches mit diesem Buchstaben beginnt

Trend

Schnelltaste, Kombination	Beschreibung
H, R, Pos1	Zurück zur ursprüngliche Ansicht
P	Schaltet die Funktion «Achsen verschieben» ein oder aus
O	Schaltet die Funktion «In Rechteck zoomen» ein oder aus
S, Strg+S	Öffnet einen Dialog zum Speichern der momentanen Abbildung in verschiedenen Grafikformaten
MAUS LINKS	Wenn Funktion «Achsen verschieben» ein: - Achsen in beliebiger Richtung verschieben Wenn Funktion «In Rechteck zoomen» ein: - In gezeichnetes Rechteck hineinzoomen
MAUS RECHTS	Wenn Funktion «Achsen verschieben» ein: - Achsenskalierung vergrößern oder verkleinern Wenn Funktion «In Rechteck zoomen» ein: - Aus gezeichnetem Rechteck herauszoomen
X+Verschieben/Zoomen	«Achsen verschieben» / «In Rechteck zoomen» auf die X-Achse beschränken
Y+Verschieben/Zoomen	«Achsen verschieben» / «In Rechteck zoomen» auf die Y-Achse beschränken

Anhang

Glossar

Begriff	Beschreibung
SILworX	Programmierwerkzeug für HIMax, HIQuad X und HIMatrix Steuerungen
API	Application Programming Interface, Programmierschnittstelle
HTTPS	Hypertext Transfer Protocol Secure, Kommunikationsprotokoll
SSL	Secure Sockets Layer, alte Bezeichnung für TLS
TLS	Transport Layer Security, Verschlüsselungsprotokoll
PEM	Privacy Enhanced Mail, Dateiformat nach RFC 7468
RSA	Rivest–Shamir–Adleman, asymmetrisches kryptographisches Verfahren
localhost	localhost benennt in der Netzwerktechnik den lokalen Rechner
SHA	Secure Hash Algorithm, kryptologische Hashfunktion
JSON	JavaScript Object Notation, Datenformat nach RFC 8259

Abbildungsverzeichnis

Bild 1:	Programm-Kontextmenü mit Menüeintrag für das Plug-In	12
Bild 2:	Anmeldedialog des Plug-Ins für den System-Login	13
Bild 3:	Initialisierung des Plug-Ins	13
Bild 4:	Hauptfenster des Plug-Ins mit Variablen-Auswahl	14
Bild 5:	Grafische Visualisierung der Prozesswerte als Funktion der Zeit.	15
Bild 6:	Schaltfläche Starten	15
Bild 7:	Schaltfläche Anhalten	16
Bild 8:	Schaltfläche Fortsetzen	16
Bild 9:	Schaltfläche Beenden	16
Bild 10:	Eingabefelder Zeitdauer Diagramm	16
Bild 11:	Schaltfläche Speichern.	16
Bild 12:	Schaltfläche Zoomen.	17
Bild 13:	Schaltfläche Verschieben.	17
Bild 14:	Schaltfläche Defaultansicht.	17

Für weitere Informationen kontaktieren Sie:

HIMA Paul Hildebrandt GmbH
Albert-Bassermann-Str. 28
68782 Brühl, Germany

Telefon +49 6202 709-0
Fax +49 6202 709-107
E-Mail info@hima.com

Erfahren Sie online mehr über HIMA Lösungen:



www.hima.com/de/